Ecossistema de Ingestão de Dados de Armazém

Relatório de Projeto

Abstract

O projeto consiste numa infraestrutura conteinerizada para a gestão de um Data Lake. O sistema automatiza a recolha, o versionamento e o armazenamento de dados de uma cadeia de retalho, integrando-os com índices de preços globais da FAO.

Diogo dos Santos Silva Mendes

20251612@iade.pt

Conteúdo

1. Resumo do Sistema 1
2. Fontes de Dados 1
3. Arquitetura da Solução 2
3.1. Camada de Origem (Source): 2
3.2. Camada de Ingestão (Ingestors): 3
3.3. Camada de Armazenamento (Data Lake): 3
4. Estratégia e Periodicidade de Ingestão 4
5. Estrutura de Diretórios HDFS 5
6. POC - Detalhes de Implementação Técnica 6
6.1. Desenvolvimento local 6
6.2. Mecanismos de Robustez 6
6.3. Fluxo de Dados (Workflow) 7
7. Conclusão e Próximos Passos 7

1. Resumo do Sistema

O projeto foca-se na Análise de Fluxo de Stock e Monitorização de Preços Agrícolas, utilizando a taxonomia da FAO (Food and Agriculture Organization). O objetivo é cruzar dados reais de preços de mercado (extraídos via API) com logs operacionais de vendas e abastecimento. Esta solução permite identificar discrepâncias entre preços globais e custos locais, otimizando a tomada de decisão na cadeia de suprimentos.

Todo o código realizado para a prova de conceito (POC) deste projeto está disponível em: https://github.com/diogosantosmendes/executivemaster_si/tree/main/tp1

2. Fontes de Dados

Para este projeto, foram integradas três fontes de dados distintas, garantindo uma cobertura completa entre dados de mercado e dados transacionais:

FONTE	TIPO	DADOS DISPONÍVEIS	FORMATO
FAOSTAT API	Real	Preços ao produtor (Producer Prices - PP) por ano, país e produto.	JSON
DB: SALES LOGS	Fictícia	Registo de vendas (ID, código FAO, zona, quantidade, valor, timestamp).	PostgreSQL
DB: SUPPLIES LOGS	Fictícia	Registo de abastecimento e entradas de stock de fornecedores.	PostgreSQL

3. Arquitetura da Solução

A solução foi desenhada utilizando uma topologia de microserviços sobre Docker, dividida em três camadas lógicas:

3.1. Camada de Origem (Source):

- **PostgreSQL 17.4:** Base de dados relacional que armazena vendas e abastecimentos em tempo real. Foi desenvolvido um seed de inicialização que gera 7.000 registos aleatórios baseados em códigos FAO reais.

	log_id [PK] integer	fao_product character varying (4)	fao_zone character varying (20)	quantity integer	value numeric (10,2)	log_time timestamp without time zone
1	1	1182	126	1	41.67	2026-04-27 07:33:21.889261
2	2	210	237	4	192.15	2026-04-26 06:23:46.323917
3	3	217	97	1	39.32	2026-05-01 07:03:32.745338
4	4	723	183	5	185.44	2026-05-01 08:02:54.271072
5	5	947	175	4	160.16	2026-04-18 07:20:51.754757
6	6	1071	53	3	47.40	2026-04-18 05:21:55.220285
7	7	1162	256	3	102.71	2026-05-02 00:36:26.673744
8	8	600	102	1	27.98	2026-04-23 00:11:25.171845
9	9	176	186	3	55.09	2026-04-20 21:16:49.100318
10	10	402	50	1	51.13	2026-05-04 16:06:50.554902
11	11	1017	215	5	27.20	2026-04-24 05:19:20.721077
12	12	101	2	2	62.28	2026-04-23 20:43:30.460294
13	13	1108	105	3	80.74	2026-04-26 05:18:11.010836
14	14	945	75	2	23.78	2026-04-23 05:02:52.294241
15	15	328	62	2	71.49	2026-04-19 04:55:36.149617
16	16	220	46	2	55.79	2026-04-25 19:33:14.268977
17	17	56	189	3	13.41	2026-04-25 13:35:17.870085
18	18	541	196	2	35.64	2026-04-24 12:15:32.414751
19	19	1111	219	1	26.44	2026-04-23 08:16:00.979113

[1]

Figure 1 - Estrutura das tabelas sales_logs e supplies_logs

- **FAOStat API [1]:** Fonte externa de dados macroeconómicos para monitorização de preços de bens de consumo.

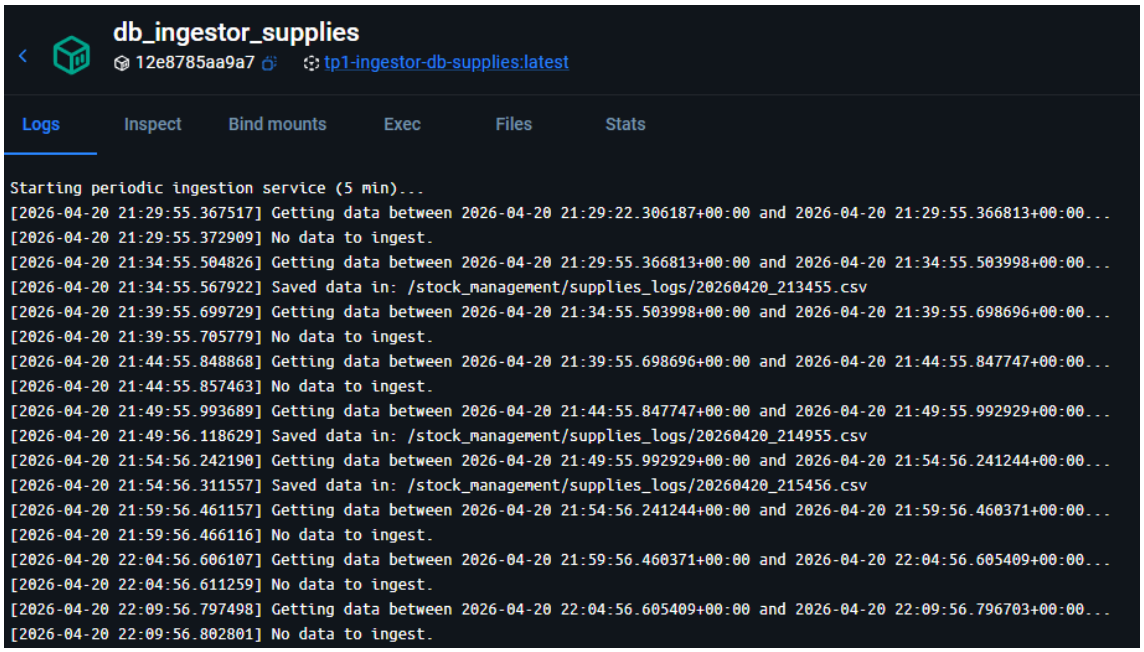

```
{
    "metadata": {…
    },
    "data": [
        {
            "Domain Code": "PP",
            "Domain": "Producer Prices",
            "Area Code": "3",
            "Area": "Albania",
            "Element Code": "5532",
            "Element": "Producer Price (USD/tonne)",
            "Item Code": "44",
            "Item": "Barley",
            "Year Code": "2024",
            "Year": "2024",
            "Months Code": "7021",
            "Months": "Annual value",
            "Unit": "USD",
            "Value": "345.1",
            "Flag": "A",
            "Flag Description": "Official figure"
        }
    ]
}
```


Figure 2 - Estrutura da resposta da API da FAO

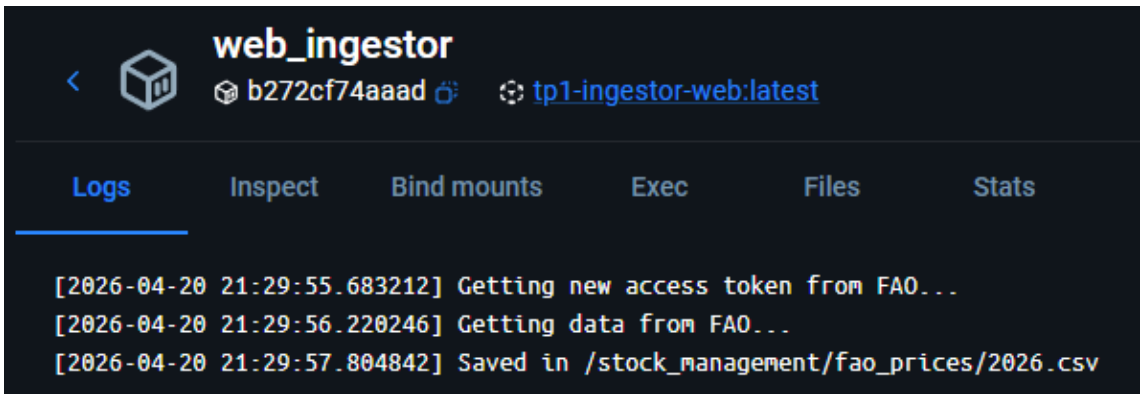
[1] https://www.fao.org/faostat/en/#developer-portal

3.2. Camada de Ingestão (Ingestors):

- **ingestor-db-<sales/supplies> (Python):** Ingestão incremental baseada em checkpoints. Monitoriza o banco de dados e extrai apenas novos registos desde a última execução.



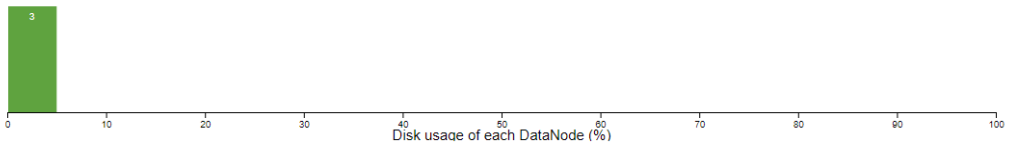
- **ingestor-web (Python):** Ingestão baseada em estado (Year-state). Consome a API da FAO apenas quando ocorre uma mudança de ano civil, otimizando o uso de recursos.



3.3. Camada de Armazenamento (Data Lake):

- **HDFS (Apache Hadoop 3.3.6):** Cluster distribuído composto por 1 Namenode e 3 Datanodes, garantindo alta disponibilidade e persistência dos dados em formato CSV estruturado por timestamps.

Datanode usage histogram



In operation

DataNode State All Show 25 entries Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Version
/default-rack/97705ec36000:9866 (172.18.0.3:9866)	http://97705ec36000:9864	1s	52m	76.15 KB	13.9 GB	1006.85 GB	6	76.15 KB (0%)	3.3.6
/default-rack/1370b6dab1a8:9866 (172.18.0.7:9866)	http://1370b6dab1a8:9864	1s	52m	52 KB	13.9 GB	1006.85 GB	2	52 KB (0%)	3.3.6
/default-rack/179f242b2a78:9866 (172.18.0.8:9866)	http://179f242b2a78:9864	1s	52m	116.06 KB	13.9 GB	1006.85 GB	4	116.06 KB (0%)	3.3.6

Showing 1 to 3 of 3 entries Previous 1 Next

4. Estratégia e Periodicidade de Ingestão

A estratégia de ingestão foi desenhada para ser eficiente e resiliente, utilizando o conceito de carga incremental:

- Ingestão Web (API FAO):
  - Periodicidade: Mensal (ou anual, dependendo da atualização da FAO).
  - Lógica: O script realiza a autenticação via *JWT*. Utiliza um mecanismo de controlo (last_processed_year.txt) para verificar se os dados de um determinado ano já foram ingeridos, evitando duplicados e chamadas desnecessárias à API.
- Ingestão de Base de Dados (Sales & Supplies):
  - Periodicidade: A cada 5 minutos (INGESTOR_PERIODICITY=300).
  - Lógica: Implementação de um ingestor Python que utiliza *checkpoints* de tempo. Apenas registos com log_time superior à última sincronização de sucesso são extraídos e movidos para o HDFS.

Para efeitos de demonstração da solução o trigger da ingestão de dados da web foi manual e o da ingestão da base de dados foi feito pode meio de um script de python.

Para uma integração nativa em ambiente de produção Hadoop, poderia utilizar-se alternativamente o comando Sqoop para a importação incremental das tabelas:

```
sqoop import \
  --connect "jdbc:postgresql://postgres-db:5432/stock_management" \
  --username "<username>" \
  --password "<password>" \
  --table "sales_logs" \
  --target-dir "/stock_management/sales_logs" \
  --incremental append \
  --check-column "log_id" \
  --last-value 0 \
  --m 1 \
  --fields-terminated-by ','
```

Algo que seria também implementado para que o sistema funciona-se de forma automatizada seria a definição de agendamentos de execução em crontab para todos os ingestores:

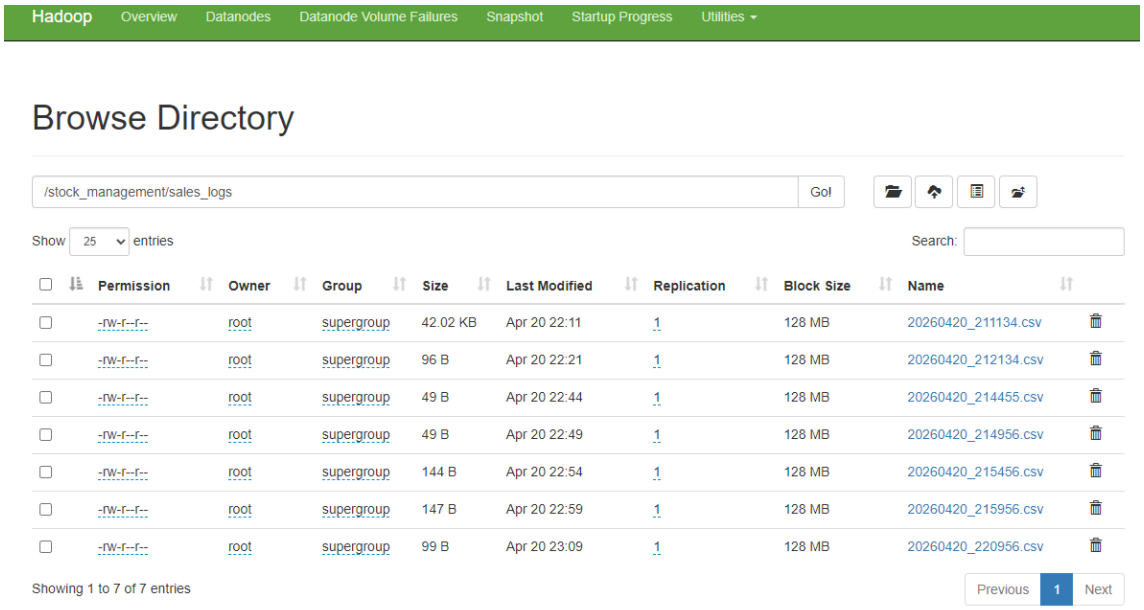
```
*/5 * * * * docker start db_ingestor_sales # run every 5 min

*/5 * * * * docker start db_ingestor_supplies # run every 5 min

0 2 * * * docker start web_ingestor # run every day at 2AM as we don't know the day that FAO updates the month or year data
```

5. Estrutura de Diretórios HDFS

A estrutura no HDFS foi organizada para facilitar a escalabilidade e o particionamento, permitindo que ferramentas como Spark ou Hive processem os dados eficientemente:

Esquema de Diretórios:

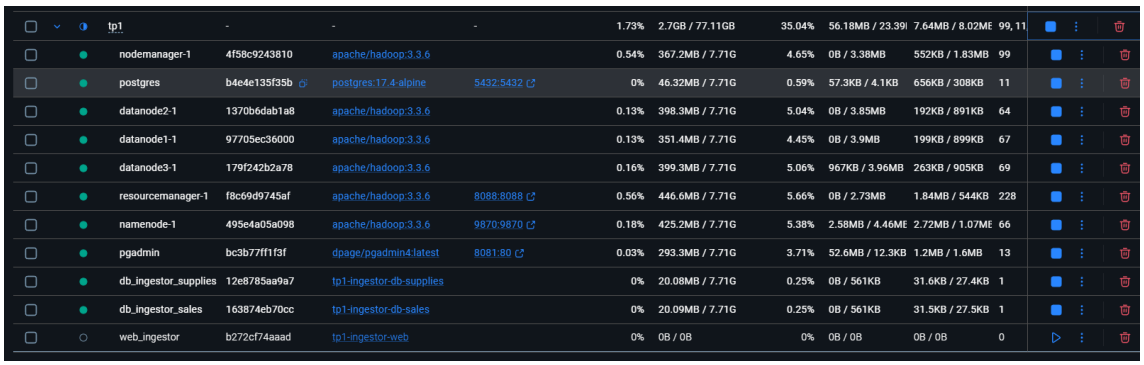
- hdfs:///stock_management/
    - fao_prices/ (Ficheiros JSON anuais vindos da API)
        - 2024.json
        - 2025.json
    - sales_logs/ (Ficheiros CSV incrementais)
        - 20260420_211500.csv
    - supplies_logs/ (Ficheiros CSV incrementais)
        - 20260420_211500.csv



6. POC - Detalhes de Implementação Técnica

6.1. Desenvolvimento local

Para efeitos de prova de conceito foi desenvolvido um Docker-compose que monta toda a infraestrutura necessária ao cenário. A base de dados relacional (PostgreSQL), O ambiente Apache Hadoop e os scripts de python para os ingestors.



6.2. Mecanismos de Robustez

1. **Checkpoints Persistentes:** Utilização de volumes Docker (./volumes/) para garantir que os ficheiros de estado (last_sync_<db_name>.txt e last_processed_year.txt) sobrevivam a reinicializações dos containers.

2. **Back-off & Retry:** O uso de restart: always no Docker Compose garante que falhas de rede temporárias com o Hadoop ou com o Postgres não interrompam o serviço permanentemente.

3. **Segurança e Configuração:** Centralização de credenciais num ficheiro .env e isolamento de rede através da market_network.

6.3. Fluxo de Dados (Workflow)

1. O **Postgres** é populado via initdb.d com um seed de 7000 registos simulando um uma cadeia de retalhos com as respetivas vendas e abastecimentos.
2. O **db_ingestor_<sales/supplies>** executa a cada 5 minutos, lê o checkpoint, extrai novos movimentos e grava no HDFS.
3. O **web_ingestor** acorda diariamente, verifica se o ano atual já foi processado; se não, autentica-se na FAO, descarrega os preços e atualiza o estado.
4. O **pgAdmin** e a interface Web do **Namenode** (porta 9870) permitem a monitorização visual de todo o processo.

7. Conclusão e Próximos Passos

O projeto demonstra o ciclo completo de Engenharia de Dados (ETL). A fundação está pronta para a próxima fase: Analytics.

Algo a explorar no futuro seria implementar um container com Apache Spark para analisar os dados de sales_logs e supplies_logs fazendo a contabilização de stock em armazém e cruzar com os preços da FAO para calcular o valor monetário total do stock em armazém em tempo real.